Society of Petroleum Engineers

SPE-229244-MS

Fast, Scalable, and Memory-Efficient High-Performance Graph Neural Networks for Subsurface in Porous Media Simulations using Multi-GPU Parallelism

Zeeshan Tariq, Physical Science and Engineering Division, King Abdullah University of Science and Technology; Mohsin Shaikh, Supercomputing Lab, KSL, King Abdullah University of Science and Technology

Copyright 2025, Society of Petroleum Engineers DOI 10.2118/229244-MS

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Abstract

Numerical simulation of underground CO_2 storage requires accurate, fast and field-scale calculations of saturation and reservoir pressures, yet high-fidelity physics-based simulators are computationally expensive to do a rapid analysis. Recent advances in deep learning, especially Graph Neural Networks (GNNs), offer promising alternatives by approximating simulation outputs while dramatically reducing computational costs by minimizing the total reliance on traditional simulators. However, training GNN models on large-scale, spatiotemporal datasets remains a significant bottleneck when constrained to a single GPU. To address the computational bottleneck of training GNN on large-scale and high-resolution datasets, we propose to use multi-GPU scalable training of GNNs. The U-Net Enhanced Graph Neural Network (U-GCN) specifically was adopted to predict the spatiotemporal evolution of gas saturation and pressure buildup in CO_2 storage in saline aquifers. The UGCN model was trained across multiple GPUs, allowing for concurrent gradient computations and parameter updates on separate GPU processes. The model operates natively on regular reservoir meshes to learn spatiotemporal evolution of pressure and saturation from thousands of simulation cases, and the training pipeline couples process-per-GPU data parallelism with a node-local caching strategy that deamplifies the source data file (in Hierarchical Data Format HDF5 format) Input-Output (I/O) by performing a single deserialization per node and broadcasting preprocessed graphs to sibling ranks. This eliminates multi-node startup contention and renders runs compute bound. Scaling experiments from 1 to 16 GPUs demonstrate substantial wall-clock reductions while preserving validation behavior. With per-GPU batch size 100, total training time decreased from 955.46 s (1 GPU) to 145.17 s (16 GPUs), a 6.58× speedup (≈41% parallel efficiency). With per-GPU batch size 8, time decreased from 5519.00 s to 1047.17 s, a 5.27× speedup (≈33% efficiency). The speedups are reported per epoch. Cross-node overheads relative to ideal doubling were ~11–35%, more pronounced at small per-GPU batches where collective latency dominates. Training and validation losses rose modestly as GPU count increased under weak scaling—consistent with the larger global batch and fewer optimizer updates per epoch—yet validation tracked training across all settings, indicating stable generalization and no data leakage. These effects are mitigated by standard large-batch remedies (e.g., linear learning-rate scaling with warmup, longer schedules,

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

optional gradient accumulation). By integrating domain-aware batching, multi-GPU data parallelism, mixed precision, and node-local caching with a GNN tailored to structured meshes, the proposed approach delivers a reproducible, high-throughput storage surrogate training regime that scales to multi-node clusters without compromising model validity. The resulting surrogates enable rapid scenario screening and outer-loop workflows (optimization and UQ) at orders-of-magnitude lower training cost compared to conventional GNN training on single a GPU baseline.

Keywords: Graph Neural Network, Multiple GPUs, Underground Gas Storage, Accelerated Training

Introduction

Surrogate models for geological storage of gases are fast, data-driven or reduced-physics emulator that approximates high-fidelity multiphase, multicomponent reservoir simulations to enable rapid forecasting, optimization, and uncertainty analysis (Chen et al., 2020; Esfandi et al., 2024; Gasós et al., 2023; Meng et al., 2024; Tang et al., 2022; Xing et al., 2023). Built from ensembles of compositional runs (e.g., spanning permeability, porosity, depth, temperature, brine salinity, well/perforation configuration, and injection schedules), the surrogate learns mappings to key responses such as plume extent, saturation fields, pressure buildup, caprock stress indicators, and trapping partitions (Tariq et al., 2025a, 2023a, 2021). Among these, Graph Neural Networks (GNNs) are particularly well suited for subsurface flow because they preserve the relational structure that governs connectivity and transmissibility (Tariq et al., 2025b, 2024). In a reservoir graph, nodes represent control volumes characterized by static attributes (e.g., porosity, permeability, depth, facies indicators) and dynamic states (e.g., pressure, CO_2 and brine saturations), while edges encode adjacency and flow capacity (Jiang, 2024; Jiang and Guo, 2023; Ju et al., 2024; Tariq et al., 2025c). Through message passing, GNNs aggregate local information to form multi-scale representations that capture both sharp saturation fronts and broad pressure support. Coupled with temporal encoders or autoregressive unrolling, GNNs can approximate spatiotemporal evolution at inference costs that are orders of magnitude lower than those of full-order simulators, enabling near-real-time scenario evaluation and screening (Jiang and Luo, 2022; Skarding et al., 2021; Wu et al., 2021; Zhou et al., 2020, 2022). However, building GNN surrogates that are sufficiently accurate and stable for subsurface storage applications presents a distinct systems challenge: training at scale on large spatiotemporal datasets. Realistic corpora consist of thousands of simulation cases spanning wide geological priors, operational controls, and monitoring horizons. Graph sizes vary across scenarios due to meshing choices and local refinement; sparsity patterns differ with well configurations and structural complexity; and feature dimensionality grows as petrophysical, and operational metadata are incorporated. In a single-GPU setting, these characteristics quickly induce two hard constraints. First, wall-clock training time becomes prohibitive, hindering rapid iteration on architectures, loss functions, and data curation. Second, device memory limits restrict batch size and temporal context, curtailing model depth and representational capacity precisely when long-horizon stability is most needed. Figure 1 shows the pros and cons of training in Multi-GPU parallelism approach.

Addressing these issues requires co-design of the model architecture, the data pipeline, and the distributed training stack. We consider a U-Net–enhanced Graph Convolutional Network (U-GCN). The spatial operator employs message-passing layers to encode local connectivity, while U-Net-style skip connections preserve high-frequency information and facilitate optimization in deeper networks. On the systems side, the training regimen emphasizes data-parallel optimization to transform throughput and memory efficiency. First, domain-aware graph mini-batching combines cluster-based pre-partitioning with controlled layer-wise neighbor sampling, reducing cut edges and moderating neighbor explosion to stabilize per-step compute. Second, dynamic bucketing groups subgraphs by approximate node/edge counts so that each rank processes similarly shaped batches, mitigating stragglers and improving kernel efficiency. Third, an asynchronous input pipeline performs subgraph extraction, feature packing, and pinned-memory staging on the host, overlapping I/O with device compute and tuning prefetch depth to saturate interconnect bandwidth

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

without exhausting host resources. The novelty of our approach lies in the integration of reservoir physics, high-resolution spatiotemporal data, and advanced GNN-based surrogate modeling into a unified, scalable workflow for underground subsurface simulation. We demonstrate that the proposed U-GCN, trained on 4000+ scenarios, achieves near-physics accuracy while providing over three orders of magnitude faster inference compared to traditional simulators.

This paper addresses the intersection of scientific need (fast, accurate prediction for underground gas/CO_2 storage), modeling choice (graph neural networks with U-Net-style skip connections), and systems design (multi-GPU data-parallel training optimized for large spatiotemporal graphs). Our main contributions are: (1) a scalable, memory-efficient training pipeline that blends domain-aware batching, mixed precision, activation checkpointing, and communication overlap; (2) an architecture and loss design suited for CO2 storage problem; and (3) an empirical demonstration that these systems choices yield state-of-the-art accuracy at orders-of-magnitude lower inference cost than full-order simulators, with substantial reductions in wall-clock training time compared to single-GPU baselines.

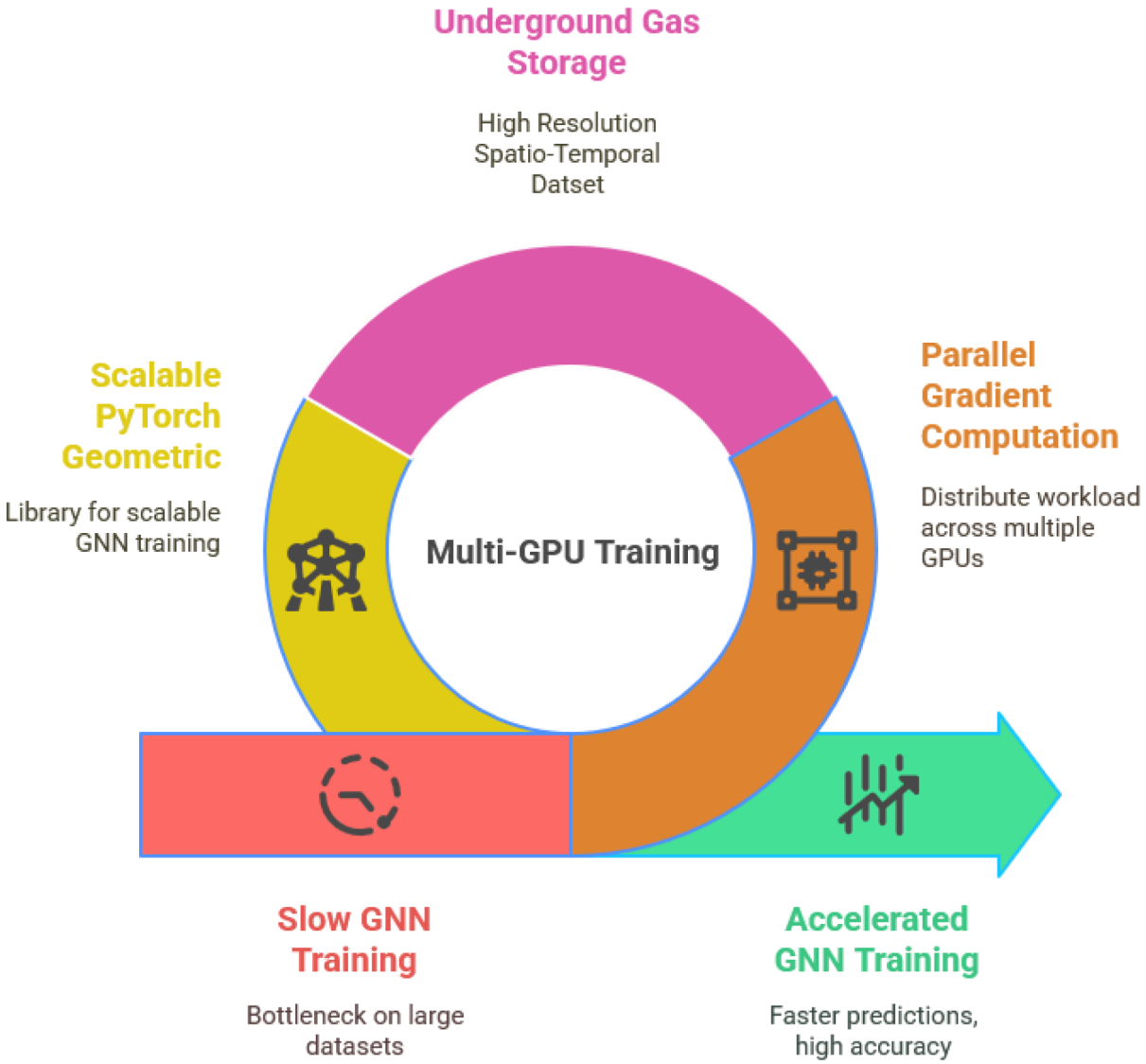

Figure 1—Accelerated GNN training with multi-GPU parallelism implementation.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

Methodology

Data source and simulation setup

We use the CO_2–brine simulation dataset from our previous publication (Tariq et al., 2023b), which was generated with a commercial compositional simulator (CMG, 2022). The flow equations were solved using a finite-difference adaptive-implicit scheme; fluid properties followed a Peng–Robinson EOS, and CO_2 viscosity was computed with the Jossi et al. (1962) correlation. The dataset comprises of a heterogeneous radial tank with a single central injector, logarithmic radial gridding, and no-flow top/bottom boundaries. The outer boundary is treated as infinite-acting via a pore-volume modifier. Supercritical CO_2 is injected at constant rates (0.2–2.0 Mt yr^{-1}) for 30 years, followed by shut-in and plume migration monitoring for 170 years. Later, the dataset was preprocessed to normalize input features (e.g., logarithmic scaling for permeability) and output variables (saturation and pressure) to the range [0, 1] for compatibility with the deep learning framework.

The dataset was split into 80% training, 10% validation, and 10% blind testing to facilitate model development and evaluation. The simulation outputs (CO_2 saturation and pressure fields) served as the ground truth for training the UGCN model. Input features included the 2D reservoir property grids (heterogeneous permeability, pore volume, reservoir temperature, reservoir salinity, initial reservoir pressure), and operational parameters (perforation thickness, injection rate).

Governing Equations

The numerical simulations were governed by the mass conservation and Darcy's law equations for two-phase flow in a porous medium. The mass conservation equation for each phase (CO_2 and brine) is given by:

$$\frac{\partial}{\partial_t}\left(\phi\rho_\alpha S_\alpha\right)+\nabla\cdot\left(\rho_\alpha u_\alpha\right)=q_\alpha \tag{1}$$

where ϕ is the porosity, ρ_α is the density of the phase α, S_α is the saturation of the phase α, u_α is the phase velocity, and q_α is the source or sink term. The phase velocity is described by Darcy's law:

$$u_\alpha=-\frac{kk_{r\alpha}}{u_\alpha}\left(\nabla P_\alpha-\rho_\alpha g\right) \tag{2}$$

where k is the absolute permeability, $k_{r\alpha}$ is the relative permeability of phase α, u_α is the viscosity, P_α is the phase pressure, and **g** is the gravitational acceleration vector. The phase pressures are related through the capillary pressure:

$$P_{CO_2}-P_{\text{brine}}=P_c\left(S_{CO_2}\right) \tag{3}$$

where $\boldsymbol{P}_c$ is the capillary pressure, modeled using the Brooks-Corey model:

$$P_c\left(S_{CO_2}\right)=P_e\left(\frac{S_{CO_2}-S_{r,CO}}{1-S_{r,CO}-S_{r,\text{brine}}}\right)^{-1/\lambda} \tag{4}$$

with P_e as the entry pressure, S_{r,CO_2} and $S_{r,brine}$ as residual saturations, and λ as the pore size distribution index. Relative permeabilities were modeled using Corey's formulation:

$$k_{r,CO_2}=k^0_{r,CO_2}\left(\frac{S_{CO_2}-S_{r,CO_2}}{1-S_{r,CO_2}-S_{r,\text{brine}}}\right)^{nCO_2} \tag{5}$$

$$k_{r,\text{brine}}=k^0_{r,\text{brine}}\left(\frac{1-S_{CO_2}-S_{r,CO}}{1-S_{r,CO_2}-S_{r,\text{brine}}}\right)^{n_{\text{brine}}} \tag{6}$$

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

where k^0_{r,CO_2} and $k^0_{r,\text{brine}}$ brine is endpoint relative permeabilities, and n_{CO_2} and n_{brine} are Corey exponents. These equations were solved numerically using a fully implicit finite-difference scheme to ensure stability in handling the complex geometries and nonlinearities introduced by faults and fractures

Results and Discussions

Distributed training and data pipeline

We train a U-Net–enhanced Graph Convolutional Network (U-GCN). The detail architecture of the UGCN model is given in our previous publication (Tariq et al., 2025c). The model uses encoder–decoder message passing with skip connections to preserve high-frequency spatial structure. The loss is a weighted sum of MSE terms for pressure and saturation over multi-step horizons. Training was performed with PyTorch Distributed Data Parallel (DDP) (Imambi et al., 2021) using the NVIDIA Collective Communications Library (NCCL) (Huang, 2020) backend (one process per GPU). Data were sharded per epoch via Distributed Sampler. To eliminate multi-node, I/O contention, we employ node-local caching: local rank 0 on each node loads and preprocesses source data file (HDF5) at once, writes an atomic cache, and sibling ranks barrier-sync and load from the cache. This de-amplifies concurrent source data file reads and removes startup jitter. The design reduces redundant opens of source file and reads from N processes to ≈ N / GPUs_per_node, which we found eliminated the long tail of startup stalls before the first training step on multi-node jobs. Figure 2 illustrates a reservoir simulation graph is partitioned into subgraphs, each containing nodes and edges that preserve local connectivity and flow relationships. These subgraphs are distributed across multiple GPUs, where each GPU processes a partition concurrently.

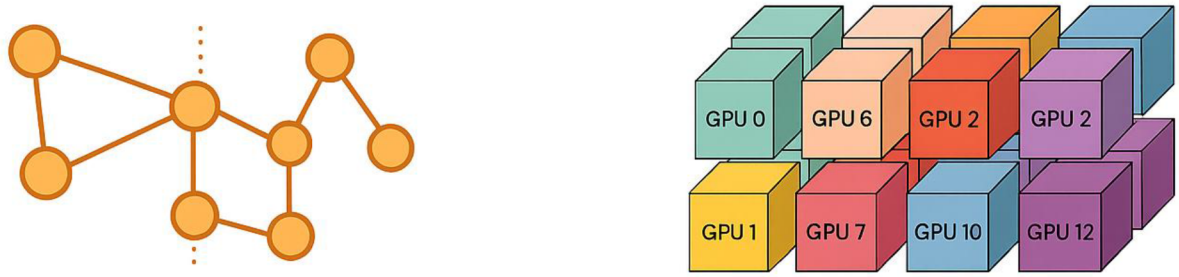

Figure 2—Schematic illustration of distributed graph training on multi-GPU clusters, modified from (PyTorch Geometric, 2022)

Scaling performance

We evaluated throughput by measuring total wall-clock time for a fixed training run as the number of GPUs increased from 1 to 16 and the job expanded from a single node to four nodes. Unless otherwise stated, "batch size" below denotes the per-GPU mini-batch (per-rank) size; thus, increasing GPU count increases the global batch (weak scaling). Table 1 provides the summary of the results. Figures 3–5 show the multi-GPU scaling performance in terms of total training time, speedup, and parallel efficiency, respectively.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

Table 1—Summary of the results against each epoch.

Nodes	GPUs	Batch size	Total time (s)	Training loss	Validation loss
1	1	100	955.46	0.014851	0.013881
1	2	100	547.53	0.019867	0.018535
1	4	100	437.14	0.026605	0.025325
2	8	100	242.67	0.029655	0.028483
4	16	100	145.17	0.036656	0.035916
1	1	8	5519.00	0.010873	0.009789
1	2	8	2891.18	0.011497	0.011597
1	4	8	2705.23	0.012364	0.011510
2	8	8	1815.32	0.013578	0.011913
4	16	8	1047.17	0.017299	0.016101

Per-GPU batch size = 100 (weak scaling)

Relative to the 1-GPU baseline (955.46 s), increasing the number of GPUs yielded the following strong speedups S and parallel efficiencies E=S/GPUs:

- 2 GPUs: 547.53 s → 1.75× speedup (87.3% efficiency), −42.7% time.
- 4 GPUs: 437.14 s → 2.19× speedup (54.6% efficiency), −54.2% time.
- 8 GPUs (2 nodes): 242.67 s → 3.94× speedup (38.1% efficiency), −74.6% time.
- 16 GPUs (4 nodes): 145.17 s → 6.58× speedup (41.1% efficiency), −84.8% time.

Cross-node scaling overhead (relative to ideal halving when doubling GPU count) was modest at this batch size: ≈11.0% overhead from 4→8 GPUs and ≈19.6% from 8→16 GPUs, attributable to additional communication and synchronization across nodes. The higher efficiency at 16 GPUs versus 8 GPUs (41.1% vs. 38.1%) reflects improved compute/communication overlap at larger bucket sizes in our configuration.

Per-GPU batch size = 8 (weak scaling)

At a smaller per-GPU batch, communication/compute ratios worsen, as expected. Relative to the 1-GPU baseline (5519.00 s):

- 2 GPUs: 2891.18 s → 1.91× speedup (95.5% efficiency), −47.6% time.
- 4 GPUs: 2705.23 s → 2.04× speedup (51.0% efficiency), −51.0% time.
- 8 GPUs (2 nodes): 1815.32 s → 3.04× speedup (38.0% efficiency), −67.1% time.
- 16 GPUs (4 nodes): 1047.17 s → 5.27× speedup (32.9% efficiency), −81.0% time.

Cross-node overheads were larger at this smaller batch size: ≈34.2% (4→8 GPUs) and ≈15.4% (8→16 GPUs), consistent with collective latency becoming a larger fraction of step time when per-rank compute is light. For both regimes, DDP delivered substantial wall-clock reductions up to 16 GPUs (−85% for batch=100 and −81% for batch=8). Larger per-GPU batches yielded better scaling (e.g., 41.1% vs. 32.9% efficiency at 16 GPUs), indicating that compute/communication balance, not raw FLOPs, governs parallel effectiveness for graph workloads.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

Convergence and generalization

Training and validation losses increased monotonically with GPU count in both batch regimes:

- Batch=100: training loss rose from 0.0149 (1 GPU) to 0.0367 (16 GPUs); validation from 0.0139 to 0.0359.
- Batch=8: training loss rose from 0.0109 (1 GPU) to 0.0173 (16 GPUs); validation from 0.0098 to 0.0161.

Two observations follow.

1. Effect of global batch size. Because the reported batch is per-GPU, the global batch grows with GPU count; at fixed epochs, this reduces the number of optimizer updates per epoch and typically necessitates learning-rate schedule adjustments (e.g., linear LR scaling with warmup) to preserve convergence. The systematic rise in both training and validation loss with more GPUs is consistent with unadjusted optimization hyperparameters under increasing global batch.
2. Small vs. large per-GPU batches. At 1 GPU, batch=8 achieved lower losses than batch=100 (train: 0.0109 vs. 0.0149; val: 0.0098 vs. 0.0139), which aligns with the generalization benefits often observed for smaller batches. However, at scale, batch=8 suffered more pronounced efficiency degradation; thus, there is a clear accuracy–throughput trade-off between small and large per-GPU batches.

The validation loss tracked training loss closely in all settings, suggesting that the distributed sharding and caching did not introduce data leakage or distributional shift across ranks. We did not observe instability (e.g., NaNs) during synchronized training, indicating that mixed precision with dynamic loss scaling and DDP gradient aggregation were numerically robust for this architecture.

Communication and cross-node effects

Transitioning from single node to multi-node runs introduced expected overheads from gradient all-reduce across nodes and from additional synchronizations (e.g., sampler barriers). At batch=100, the 4→8 GPU step incurred ~11% overhead beyond ideal; at batch=8, the same step incurred ~34% overhead, indicating that per-rank work size is the dominant lever for masking interconnect latency. Gradient bucket sizing and overlapping communication with backpropagation helped, but diminishing returns appeared once per-rank computation fell below a threshold (batch=8 at 16 GPUs).

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

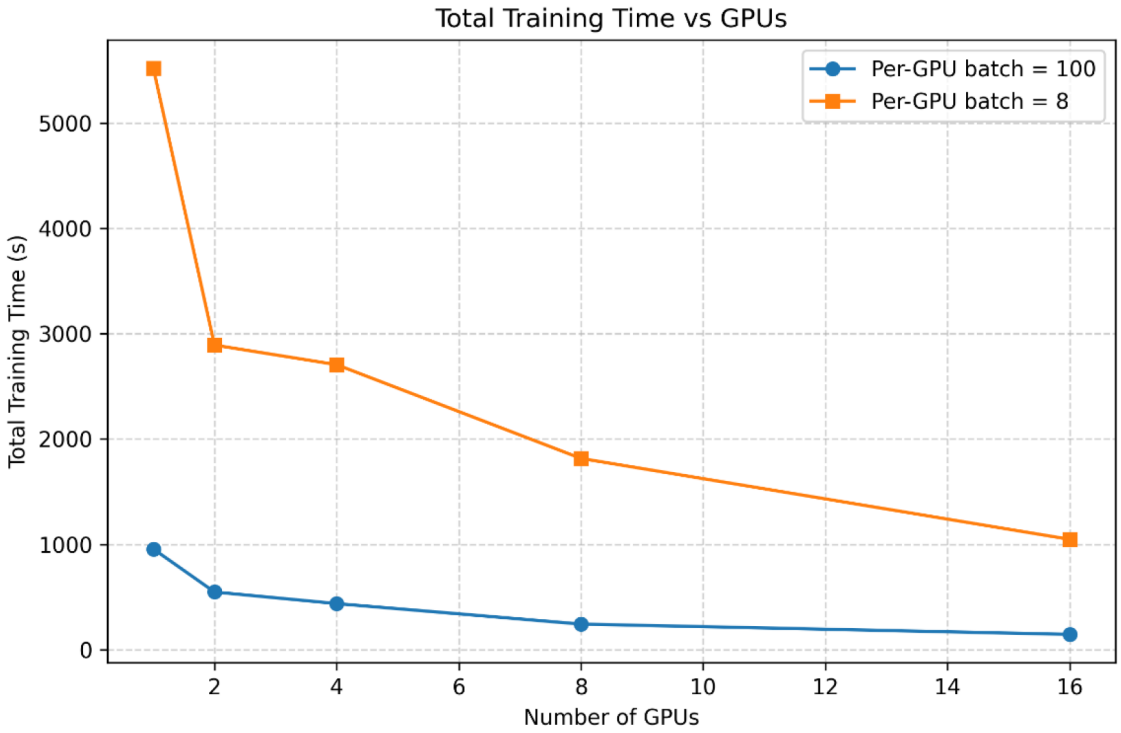

Figure 3—Comparison of total training time with the batch size of 8 and 100.

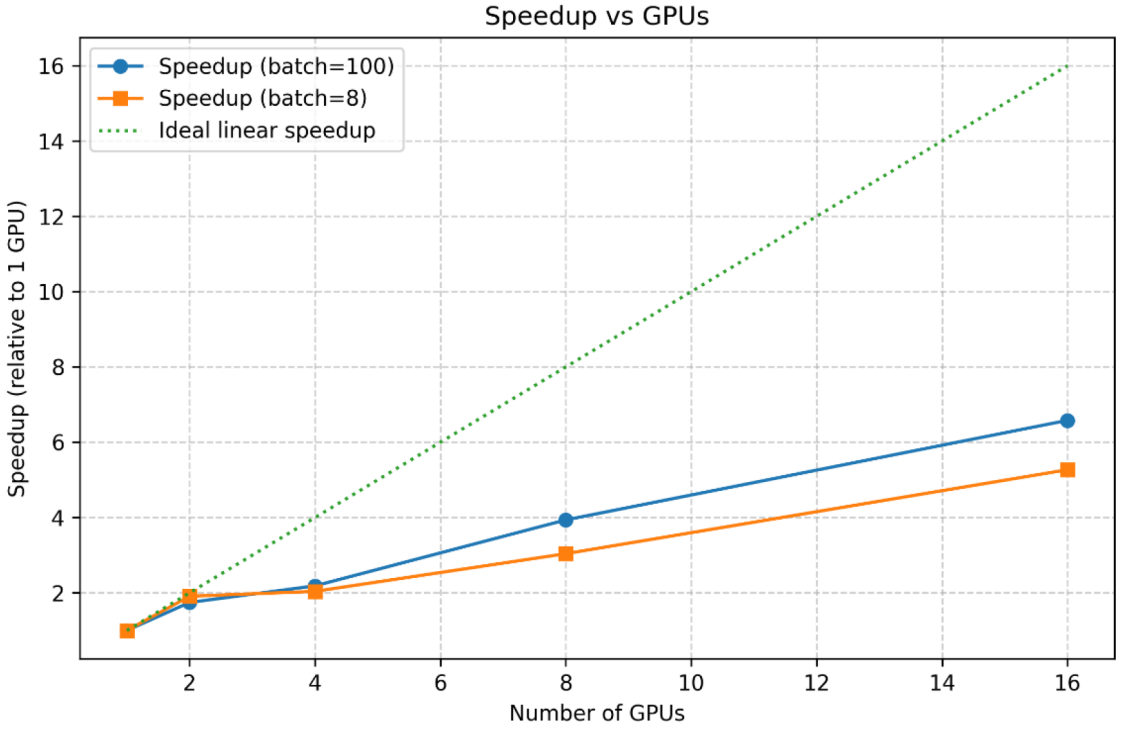

Figure 4—Comparison of Speedup with the batch size of 8 and 100.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

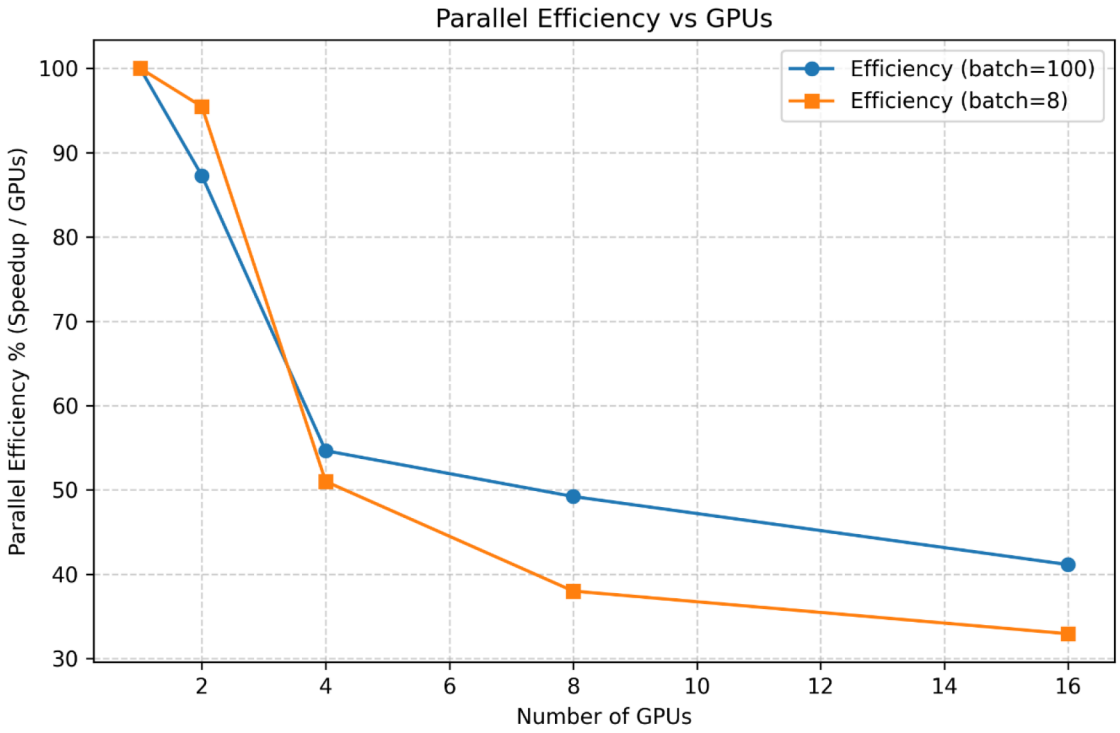

Figure 5—Comparison of parallel efficiency with the batch size of 8 and 100.

Summary and Conclusions

DDP training with NCCL, paired with a node-local caching pipeline, enabled efficient multi-GPU scaling for the CO_2-storage GNN surrogate while maintaining stable generalization. Based on the results the following inferences can be made:

- End-to-end training time dropped from 955.46 → 145.17 s at 16 GPUs with per-GPU batch=100 (6.58×, ~41% efficiency) and from 5519.00 → 1047.17 s with batch=8 (5.27×, ~33%). Speedups were sublinear, as expected for synchronized training.
- Larger per-GPU batches improved compute/communication balance and scaling efficiency; smaller batches produced lower losses but scaled less effectively. Selecting an operating point should balance efficiency (larger per-GPU batches) against convergence (smaller effective batches).
- Moving from single- to multi-node introduced ~11–35% overhead beyond ideal doubling, with higher penalties at small per-GPU batches where collective latency dominates.
- Under weak scaling (global batch grows with GPU count), both training and validation losses rose modestly. This is consistent with large-batch effects and can be mitigated via linear LR scaling with warmup, longer schedules to equalize update counts, and optional gradient accumulation/clipping. Validation tracked training across all settings, indicating no data leakage; mixed precision with dynamic loss scaling remained numerically stable.
- Node-local caching removed multi-node I/O contention and startup jitter by de-amplifying HDF5 reads to one deserialization per node, rendering runs compute-bound once caches were established.

Acknowledgement

Authors would like to acknowledge Supercomputing lab (KSL) at King Abdullah University of Science and Technology to run GNN codes.

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

List of Symbols

DDP	Distributed Data Parallel
EOS	Equation of State
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HDF5	Hierarchical Data Format
MSE	Mean Squared Error
NANs	Not a Number
NCCL	NVIDIA Collective Communications Library
UGCN	U-Net Enhanced Graph Convolutional Network

References

Chen, M., Abdalla, O.A., Izady, A., Reza Nikoo, M., Al-Maktoumi, A., 2020. Development and surrogate-based calibration of a CO2 reservoir model. *J. Hydrol*. 586. https://doi.org/10.1016/j.jhydrol.2020.124798

Esfandi, T., Sadeghnejad, S., Jafari, A., 2024. Effect of reservoir heterogeneity on well placement prediction in CO2-EOR projects using machine learning surrogate models: Benchmarking of boosting-based algorithms. *Geoenergy Sci. Eng*. 233. https://doi.org/10.1016/j.geoen.2023.212564

Gasós, A., Becattini, V., Brunetti, A., Barbieri, G., Mazzotti, M., 2023. Process performance maps for membrane-based CO2 separation using artificial neural networks. *Int. J. Greenh. Gas Control* 122. https://doi.org/10.1016/j.ijggc.2022.103812

Huang, J., 2020. NVIDIA Collective Communications Library [WWW Document]. NVIDIA Corp. URL https://developer.nvidia.com/nccl

Imambi, S., Prakash, K.B., Kanagachidambaresan, G.R., 2021. *PyTorch, in: EAI/Springer Innovations in Communication and Computing*. pp. 87–104. https://doi.org/10.1007/978-3-030-57077-4_10

Jiang, J., 2024. Simulating multiphase flow in fractured media with graph neural networks. *Phys. Fluids* 36. https://doi.org/10.1063/5.0189174

Jiang, J., Guo, B., 2023. *Graph Convolutional Networks for Simulating Multi-phase Flow and Transport in Porous Media*.

Jiang, W., Luo, J., 2022. Graph neural network for traffic forecasting: A survey. *Expert Syst. Appl*. https://doi.org/10.1016/j.eswa.2022.117921

Ju, X., Hamon, F.P., Wen, G., Kanfar, R., Araya-Polo, M., Tchelepi, H.A., 2024. Learning CO2 plume migration in faulted reservoirs with Graph Neural Networks. *Comput. Geosci*. 193. https://doi.org/10.1016/j.cageo.2024.105711

Meng, S., Fu, Q., Tao, J., Liang, L., Xu, J., 2024. Predicting CO2-EOR and storage in low-permeability reservoirs with deep learning-based surrogate flow models. *Geoenergy Sci. Eng*. 233. https://doi.org/10.1016/j.geoen.2023.212467

PyTorch Geometric, 2022. *PyG Documentation — pytorch_geometric documentation [WWW Document]*. PyTorch. URL https://pytorch-geometric.readthedocs.io/en/latest/ (accessed 8.28.25).

Skarding, J., Gabrys, B., Musial, K., 2021. Foundations and Modeling of Dynamic Networks Using Dynamic Graph Neural Networks: A Survey. *IEEE Access* **9**, 79143–79168. https://doi.org/10.1109/ACCESS.2021.3082932

Tang, M., Ju, X., Durlofsky, L.J., 2022. Deep-learning-based coupled flow-geomechanics surrogate model for CO2 sequestration. *Int. J. Greenh. Gas Control* 118. https://doi.org/10.1016/j.ijggc.2022.103692

Tariq, Z., Abu-Al-Saud, M.O., Mahmoud, M., Zhu, C., Sun, S., Yan, B., 2025a. Fast tracking of safe CO2 trapping indices using machine learning for smarter reservoir management. *Petroleum*. https://doi.org/10.1016/j.petlm.2025.01.002

Tariq, Z., Abualsaud, M., He, X., AlMajid, M., Sun, S., Hoteit, H., Yan, B., 2025b. A U-Net Enhanced Graph Neural Network to Simulate Geological Carbon Sequestration. *SPE J*. **30**, 3950–3968. https://doi.org/10.2118/220757-PA

Tariq, Z., Abualsaud, M., He, X., AlMajid, M., Sun, S., Hoteit, H., Yan, B., 2025c. A U-Net Enhanced Graph Neural Network to Simulate Geological Carbon Sequestration. *SPE J*. 1–19. https://doi.org/10.2118/220757-PA

Tariq, Z., Aljawad, M.S., Hasan, A., Murtaza, M., Mohammed, E., El-Husseiny, A., Alarifi, S.A., Mahmoud, M., Abdulraheem, A., 2021. A systematic review of data science and machine learning applications to the oil and gas industry. *J. Pet. Explor. Prod. Technol*. **11**, 4339–4374. https://doi.org/10.1007/s13202-021-01302-2

Tariq, Z., Gudala, M., Yan, B., Sun, S., Rui, Z., 2023a. *Optimization of Carbon-Geo Storage into Saline Aquifers: A Coupled Hydro-Mechanics-Chemo Process*. https://doi.org/10.2118/214424-MS

Tariq, Z., Hoteit, H., Sun, S., Abualsaud, M., He, X., AlMajid, M., Yan, B., 2024. The U-Net Enhanced Graph Neural Network for Multiphase Flow Prediction: An Implication to Geological Carbon Sequestration, in: SPE Annual Technical Conference and Exhibition. SPE. https://doi.org/10.2118/220757-MS

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025

Tariq, Z., Yan, B., Sun, S., 2023b. Physics Informed Surrogate Model Development in Predicting Dynamic Temporal and Spatial Variations During CO2 Injection into Deep Saline Aquifers. Soc. Pet. Eng. - SPE Reserv. Characterisation Simul. Conf. Exhib. 2023, RCSC 2023. https://doi.org/10.2118/212693-MS

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S., 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Trans. Neural Networks Learn. Syst*. **32**, 4–24. https://doi.org/10.1109/TNNLS.2020.2978386

Xing, L., Jiang, H., Wang, S., Pinfield, V.J., Xuan, J., 2023. Data-driven surrogate modelling and multi-variable optimization of trickle bed and packed bubble column reactors for CO2 capture via enhanced weathering. *Chem. Eng. J*. 454. https://doi.org/10.1016/j.cej.2022.139997

Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M., 2020. Graph neural networks: A review of methods and applications. *AI Open*. https://doi.org/10.1016/j.aiopen.2021.01.001

Zhou, Y., Zheng, H., Huang, X., Hao, S., Li, D., Zhao, J., 2022. Graph Neural Networks: Taxonomy, Advances, and Trends. *ACM Trans. Intell. Syst. Technol*. https://doi.org/10.1145/3495161

Downloaded from https://onepetro.org/SPEADIP/25ADIP/conference/25ADIP by Dmitry Kruglov on 09 November 2025